

AgentSkills / Agent Workflow Kit 設計書

v0.1

1. 背景

LLMエージェントは、アイデア出し、設計、仕様整理、ドキュメント作成などの「拡張フェーズ」では非常に強い。

一方で、実装後のバグ修正、テスト修正、既存コードへの小改修、差分レビューなどの「収束フェーズ」では、以下の問題が起きやすい。

- 既存仕様を壊す
- 目的外のリファクタリングを混ぜる
- テスト期待値を実装都合で変更する
- 過去の会話文脈を誤って参照する
- 途中で変更された仕様を混同する
- 変更範囲が広がる
- 失敗時に原因分析なしで連続修正する
- スキルやサブエージェントが期待通りに自動発動しない

特に、LLMはOR的な発想、つまり候補を広げる処理には強いが、AND的な処理、つまり複数の制約を同時に満たす処理には弱い。

そのため、収束フェーズでは、LLMの能力だけに頼るのではなく、以下のような制御を組み合わせる必要がある。

- AGENTS.md / CLAUDE.md による行動ルール
- Session Brief による短期状態固定
- プロンプトテンプレートによる手順固定
- git diff による実変更レビュー
- サブエージェントまたは別セッションによる独立レビュー
- shell script によるローカルゲート
- Git Hook による pre-commit / pre-push 制御
- 小さいコミットによる変更範囲の抑制

2. 目的

この設計では、既存の AgentSkills リポジトリを拡張し、Claude Code / Codex 共通で利用できる「収束フェーズ制御キット」を追加する。

目的は以下である。

1. Claude Code / Codex の両方で使える共通のエージェント運用ルールを整備する
2. スキルの自動発動に依存せず、明示的に呼び出せるプロンプト・擬似コマンドを整備する
3. 収束フェーズを SDD + TDD + diff review + gate の流れで安定化する

4. サブエージェントレビューでは親コンテキストを前提にせず、SESSION_BRIEF と git diff を根拠にする
5. Git Hook と shell script により、LLMの外側で最低限のローカルゲートを実現する
6. Skill、prompt、subagent、shell script、Git Hookの実行有無と結果を利用者が確認できる構成にする

3. 基本方針

3.1 Claude 専用にしない

リポジトリ名は AgentSkills とし、Claude Code だけでなく Codex でも利用する。

そのため、Claude固有機能だけに依存しない。

中心に置くものは以下とする。

- Markdown
- shell script
- Git Hook
- git diff
- AGENTS.md
- SESSION_BRIEF.md
- prompts/*.md

Claude Code の Skill、Subagent、Slash Command は補助として扱う。

Codex の AGENTS.md、Subagent、Slash Command、approval / sandbox も補助として扱う。

3.2 スキルの自動発動に依存しない

スキルは有用だが、期待した場面で自動発動しない場合がある。

そのため、以下の方式を採用する。

- 常時ルールは AGENTS.md / CLAUDE.md に書く
- 詳細手順は prompts/*.md に書く
- 今回の作業状態は SESSION_BRIEF.md に書く
- 実行時は擬似コマンドまたは明示指示で prompts/*.md を呼ぶ
- 強制的な検査は shell script / Git Hook に置く

v0.1の収束キット自体はClaude/Codex固有のSkillとして実装しない。既存Skillは従来どおりAgentSkillsを正本とし、各ツールのSkillディレクトリへsymlinkして利用する。将来、収束キットのSkill入口を追加する場合も、共通本文は common/ に置き、入口だけを各ツールへsymlinkする。

3.3 収束フェーズでは SDD + TDD を使う

簡単なバグ修正でも、以下の順番で進める。

1. Spec: 現在の挙動、期待仕様、不一致点を整理する
2. Test: 失敗テストを作る、または既存テストで再現する

3. Implement: テストを通すための最小差分で実装する
4. Review: git diff を確認する
5. Gate: pre-commit-gate.sh を実行する
6. Commit: 小さい単位でコミットする

3.4 レビュー担当は親コンテキストを引き継がない

レビュー担当サブエージェント、または別セッションのレビュー担当は、実装担当の会話履歴や推論を前提にしない。

レビューの根拠は以下に限定する。

- AGENTS.md
- SESSION_BRIEF.md
- git status
- git diff
- git diff --cached
- untracked files
- 必要な場合のみ対象ファイルの現在内容

レビュー担当はコード変更しない。

3.5 実行状態と結果を可視化する

利用者が、どの制御が実際に動き、どのような結果になったかを確認できるようにする。

共通の状態表現は以下とする。

- START: 実行開始
- PASS: 正常終了
- WARNING: 確認が必要だが継続可能
- BLOCKER: ポリシー違反により停止
- FAIL: scriptや実行環境自体の異常
- SKIP: 条件により未実行

Skill、prompt、subagentについては、共通基盤から自動発動を機械的に検出できない。そのため、エージェントに以下を義務付ける。

- Skillを使用するときは、開始時にSkill名を表示する
- 擬似コマンドを処理するときは、参照したpromptファイルを表示する
- SESSION_BRIEFを参照したときは、そのパスを表示する
- subagentを起動したときは、役割と参照情報を表示する
- 終了時に OK / WARNING / BLOCKER を表示する
- 実際に参照していないSkillやpromptを、発動または参照したと表現しない

shell scriptとGit Hookは、処理名、対象、判定理由、解消方法、最終結果を標準出力または標準エラーに表示する。

3.6 Triggerとルール配送

処理のTriggerは以下の4種類に分類する。

1. 自動駆動: Claude CodeまたはCodexの開始、通常依頼の受信
2. ユーザー指示・擬似コマンド: @converge-bugfix、@diff-review など
3. ローカルGit Hook: git commit、git push
4. GitHub Hookまたはイベント: Pull Request、push、workflow。v0.1では対象外

rules、prompts、SESSION_BRIEF、shell script、subagentはTriggerではなく、Triggerから起動される処理または配送される入力とする。

自動駆動では、製品が AGENTS.md または CLAUDE.md をコンテキストへロードする。Claude Codeでは CLAUDE.md から共通ルールである AGENTS.md を読むよう指示する。親LLMは AGENTS.md のMode Selectorに従い、通常依頼を Expansion / Convergence / Uncertain に分類する。

Mode Selectorの優先順位:

1. ユーザーによる明示的なモード指定
2. 作業目的
3. 仕様と期待結果の確定度
4. 既存コードや既存動作への変更か
5. 判断できない場合は Uncertain

既存不具合、失敗テスト、確定仕様との不一致、既存コードへの限定変更、デグレ、review/test/gate指摘の修正は Convergence とする。アイデア出し、要件整理、比較、未確定仕様の新機能、アーキテクチャ検討は Expansion とする。両方に該当する場合や期待仕様が不明な場合は Uncertain とし、コードを変更せずユーザーへ確認する。

判定結果は、mode、trigger、判定根拠、次に読むファイル、次のphaseとともに表示する。Convergenceへ入ったらプロジェクトルートの SESSION_BRIEF.md にモードを固定し、ユーザー指示または確認なしに別モードへ変更しない。

4. 制御レイヤー

4.1 入力・文脈制御レイヤー

LLMの振る舞いを制御する層。

対象:

- AGENTS.md
- CLAUDE.md
- prompts/*.md
- 擬似コマンド
- 通常プロンプト

役割:

- どう振る舞うかを定義する
- 収束フェーズの作業手順を固定する
- レビュー担当の禁止事項を定義する
- SDD + TDD の順序を定義する

4.2 状態固定レイヤー

今回の作業状態を固定する層。

対象:

- プロジェクトルートの SESSION_BRIEF.md
- common/briefs/*.template.md

役割:

- 今回の目的を固定する
- 現在の確定仕様を固定する
- 対象・非対象を明示する
- 禁止事項を明示する
- 過去の会話よりも SESSION_BRIEF を優先させる

4.3 LLM拡張機能レイヤー

Claude Code / Codex の機能を補助的に使う層。

対象:

- Claude Skill
- Claude Code Subagent
- Claude Code Slash Command
- Codex Subagent
- Codex Slash Command
- Model Switching

役割:

- 必要に応じて専門化された作業を呼び出す
- レビュー担当を分離する
- 拡張フェーズと収束フェーズでモデルを切り替える

注意:

- 製品固有機能には依存しすぎない
- 共通基盤は Markdown / sh / Git に置く

4.4 ローカル実行制御レイヤー

LLMの外側で機械的に確認する層。

対象:

- common/gates/pre-commit-gate.sh
- common/gates/check-sensitive-files.sh
- common/gates/check-large-files.sh
- common/gates/check-diff-basic.sh

役割:

- 秘密ファイルの混入を防ぐ
- 巨大ファイルの混入を防ぐ
- staged diff の基本確認を行う
- 空白エラーを検出する
- Git Hook から呼び出せる共通ゲートにする

4.5 Git制御レイヤー

Gitの仕組みで制御する層。

対象:

- common/hooks/pre-commit
- common/hooks/pre-push
- git diff
- git diff --cached
- git status
- 小さいcommit

役割:

- commit 前に gate を自動実行する
- main / master への直接 push を防ぐ
- 実際の差分を真実としてレビューする
- 1目的1コミットで差分を小さくする

4.6 レビュー制御レイヤー

実装担当とは別視点で確認する層。

対象:

- prompts/diff-review.md
- prompts/subagent-review.md
- Claude Code Subagent
- Codex Subagent
- 別セッションレビュー

役割:

- 目的外変更を検出する
- テスト期待値の都合のよい変更を検出する
- 対象外ファイル変更を検出する
- 実装担当の文脈に引きずられないレビューを行う

レビュー対象は以下とする。

- git diff によるunstaged差分
- git diff --cached によるstaged差分
- git status と必要なファイル確認によるuntracked files

レビュー結果後の動作は以下とする。

- OK: 次工程へ進める
- WARNING: 内容を報告し、ユーザー判断を待つ
- BLOCKER: 作業を停止する
- gateまたはtest失敗: 追加修正を行わずfailure analysisへ進む
- failure analysis後の修正: ユーザー許可を得てから行う

4.7 実行可視性レイヤー

各制御の実行状態と結果を利用者へ伝える層。

対象:

- Skillとpromptの開始・終了報告
- subagentの起動・結果報告
- shell scriptとGit Hookの実行メッセージ
- gateとreviewの判定結果

表示形式は、原則として次のプレフィックスを使用する。

```
[AgentSkills][対象種別][状態] 処理名
```

例:

```
[AgentSkills][PROMPT][START] @diff-review
Source: common/prompts/diff-review.md
[AgentSkills][REVIEW][WARNING] Test files were modified.
[AgentSkills][PROMPT][END] @diff-review
Result: WARNING
```

4.8 GitHub制御レイヤー

リモート側で最終的に守る層。

初期段階では必須ではない。

将来的な対象:

- Pull Request
- Branch protection
- Required review
- GitHub Actions
- Required checks

役割:

- local gate をすり抜けた変更をリモートで止める
- main を Prod として保護する
- PRレビューを必須化する

5. 推奨ディレクトリ構成

既存リポジトリ内に、共通エージェント運用部分を追加する。

初期案:

```
AgentSkills/  
  README.md  
  test-orchestrator/  
  code-review/  
  cowork-chrome-launcher/  
  agents/  
  common/  
    README.md  
    rules/  
      AGENTS.base.md  
      CLAUDE.base.md  
    prompts/  
      converge-bugfix.md  
      diff-review.md  
      subagent-review.md  
      failure-analysis.md  
  briefs/  
    SESSION_BRIEF.template.md  
  config/  
    AGENT_MODELS.template.md  
  gates/  
    pre-commit-gate.sh  
    check-sensitive-files.sh  
    check-large-files.sh  
    check-diff-basic.sh  
    check-llm-review.sh  
  lib/  
    review-common.sh  
  reviewers/  
    review-staged-diff.sh  
    record-manual-review.sh  
  schemas/  
    review-result.schema.json  
  hooks/  
    pre-commit  
    pre-push  
  setup/  
    setup-hooks.sh  
  tests/  
    run-tests.sh
```

5.1 既存部分

既存の以下は維持する。

```
test-orchestrator/  
code-review/  
cowork-chrome-launcher/  
agents/
```

5.2 新規追加部分

新規に common/ を追加する。

common/ は Claude / Codex 共通で使えるエージェント運用キットとする。

5.3 利用先への配置

AgentSkills内の正本は common/ とする。個人のローカル利用では、既存Skillの運用と揃えて、利用先プロジェクトの .agentskills からAgentSkillsの common/ へsymlinkする。

```
my-project/  
  .agentskills -> /path/to/AgentSkills/common  
  AGENTS.md  
  CLAUDE.md  
  SESSION_BRIEF.md  
  AGENT_MODELS.md
```

共通rules、prompts、templates、gates、hooks、setup scriptsは .agentskills/ 経由で利用する。プロジェクト固有の AGENTS.md、CLAUDE.md、SESSION_BRIEF.md、AGENT_MODELS.md はsymlinkせず、利用先プロジェクトの実ファイルとする。

利用先で common/ をリンク名にしないのは、アプリケーション側の既存 common/ ディレクトリとの衝突を避けるためである。

チーム共有や単独配布ではsymlinkが他のPCで切れる可能性があるため、コピーを代替手段としてREADMEに記載する。Git submodule、installer、テンプレート生成は将来対応とする。

6. 擬似コマンド

Claude / Codex 共通で使うため、正式な slash command ではなく、擬似コマンドを定義する。

初期コマンド:

- @converge-bugfix
- @diff-review
- @subagent-review
- @gate
- @failure-analysis

6.1 @converge-bugfix

用途:

- 収束フェーズのバグ修正を開始する

動作:

- AGENTS.md を読む
- SESSION_BRIEF.md を読む
- common/prompts/converge-bugfix.md の手順に従う
- 使用するpromptと現在のphaseを表示する
- 最初は Phase 1: Spec のみ行う
- コード変更は禁止

例:

@converge-bugfix ログイン後に /dashboard に遷移しない問題。まず Phase 1: Spec のみ。コード変更は禁止。

6.2 @diff-review

用途:

- 実装後の git diff をレビューする

動作:

- git status を確認する
- git diff、git diff --cached、untracked filesを確認する
- SESSION_BRIEF と差分を照合する
- common/prompts/diff-review.md を参照したことを表示する
- OK / WARNING / BLOCKER と、その後の動作を表示する
- コード変更は禁止

例:

@diff-review 今回の目的と無関係な変更がないか確認。コード変更は禁止。

6.3 @subagent-review

用途:

- レビュー専用サブエージェント、または別セッションで独立レビューを行う

動作:

- 親会話の文脈を前提にしない
- AGENTS.md、SESSION_BRIEF.md、git status、git diff、git diff --cached、untracked filesを根拠にする

- subagentの役割と参照情報を表示する
- OK / WARNING / BLOCKER で判定する
- コード変更は禁止

例:

```
@subagent-review AGENTS.md、SESSION_BRIEF.md、git status、git diff、git diff --cached、untracked filesを根拠にレビュー。親会話の文脈は前提にしない。コード変更は禁止。
```

6.4 @gate

用途:

- ローカル gate を実行する

動作:

- ./common/gates/pre-commit-gate.sh を実行する
- 実行したgate、各check、最終結果を表示する
- 失敗した場合は、追加修正前に原因分析する

例:

```
@gate 失敗した場合は、追加修正の前に原因分析だけしてください。
```

6.5 @failure-analysis

用途:

- gate、テスト、レビュー失敗時に原因分析を行う

動作:

- すぐに追加修正しない
- 失敗原因を整理する
- 最小修正方針を出す
- コード変更は禁止

例:

```
@failure-analysis pre-commit-gate が失敗しました。追加修正の前に原因分析のみ行ってください。
```

7. 収束フェーズ標準フロー

Step 1: Session Brief 作成

作業前にプロジェクトルートの SESSION_BRIEF.md を作成または更新する。

テンプレートは `common/briefs/SESSION_BRIEF.template.md` とする。

含める内容:

- 作業モード
- 今回の目的
- 現在の確定仕様
- 現在の問題
- 対象
- 非対象
- 禁止事項
- 検証方法

併せて作業開始時の `git status` と既存のstaged filesを確認する。既にstagedされている変更は今回の作業と分離し、勝手に追加、解除、変更しない。

Step 2: Spec

LLMに `@converge-bugfix` を指示する。

この段階ではコード変更禁止。

出力させるもの:

- 現在の挙動
- 期待仕様
- 不一致点
- 影響範囲
- テストで固定すべき条件
- 修正対象候補
- 変更予定ファイル

Step 3: Test

ユーザーが許可したらテスト作成に進む。

ルール:

- 仕様に基づいて失敗テストを作る
- 既存テストで再現できる場合は新規追加しない
- 実装コードは変更しない
- テスト期待値を実装都合で変更しない

Step 4: Implement

ユーザーが許可したら実装に進む。

ルール:

- 最小差分で実装する
- テスト期待値を勝手に変更しない
- 対象外ファイルを変更しない

- 関係ないリファクタリングをしない

Step 5: Diff Review

実装後、@diff-review を実行する。

unstaged、staged、untrackedの変更を確認対象とする。

確認観点:

- 今回の目的に必要な変更か
- 仕様と関係する変更か
- 目的外変更がないか
- 最小差分か
- テスト期待値を都合よく変更していないか

Step 6: Subagent Review

必要に応じて @subagent-review を実行する。

レビュー担当は、親会話の文脈を前提にせず、AGENTS.md、SESSION_BRIEF.md、git status、git diff、git diff --cached、untracked filesを根拠にする。

Step 7: Staging

Diff Reviewが OK になったら、エージェントが今回の目的に必要なファイルをstagingする。個別のユーザー許可は必須としない。

ルール:

- SESSION_BRIEFの対象、変更予定ファイル、git diffの内容を根拠に選ぶ
- stagingするファイルと選定理由を事前に表示する
- git add -- <path>... のように対象パスを明示する
- git add .、git add -A、対象を限定しないgit add -u は使用しない
- 既にstagedされていた変更を解除または変更しない
- 対象ファイルに目的外の変更が混在する場合はstagingせず WARNING とする
- staging後に実際にstagedされたファイルを表示する

Step 8: Staged Diff Review

git diff --cached と git status を確認し、コミット候補がSESSION_BRIEFの目的と一致することを再確認する。

確認観点:

- 必要なファイルだけがstagedされているか
- unstagedまたはuntrackedの必要な変更が取り残されていないか
- 既存のstaged変更が混在していないか
- 目的外変更やテスト期待値の都合のよい変更がないか

結果と根拠を OK / WARNING / BLOCKER で表示する。

Step 9: Gate

@gate または直接 ./common/gates/pre-commit-gate.sh を実行する。

失敗時は @failure-analysis に進む。

各工程の判定後は以下に従う。

- OK: 次工程へ進む
- WARNING: ユーザー判断を待つ
- BLOCKER: 作業を停止する
- testまたはgate失敗: 追加修正せず原因分析へ進む

Step 10: Commit

1目的1コミットで commit する。

pre-commit hook により gate が自動実行される。

8. Git Hook 方針

8.1 pre-commit

中心的に使う。

実行内容:

- git diff --cached --check
- check-sensitive-files.sh
- check-large-files.sh
- check-diff-basic.sh
- check-llm-review.sh

各scriptは次の情報を表示する。

- scriptとcheckの開始・終了
- PASS / WARNING / BLOCKER / FAIL / SKIP
- 問題となったファイル
- blockまたはwarningの理由
- サイズ制限などの判定根拠
- stagingから外す方法などの解消手順
- gate全体の最終結果

機械的checkのblockは終了コード 1、情報提供だけのwarningは終了コード 0 とする。LLM reviewの WARNING と BLOCKER はcommitを止めるため終了コード 1 とする。ファイル自体の削除や変更は行わない。

8.1.1 Codexによる自動staged diffレビュー

pre-commitは機械的checkの後に check-llm-review.sh を実行し、非対話モードのCodex CLIを独立reviewerとして起動する。ユーザーまたは親エージェントによる明示的な @diff-review 指示は必要としない。

い。

reviewerは概念的に次の条件で起動する。

```
codex exec \  
  --sandbox read-only \  
  --ephemeral \  
  --output-schema .agentskills/schemas/review-result.schema.json \  
  --cd "$repo_root" \  
  -
```

reviewerは AGENTS.md、SESSION_BRIEF.md、.agentskills/prompts/subagent-review.md、git status、git diff --cached を根拠にする。親会話の履歴や実装担当の推論は渡さない。コード、staging、SESSION_BRIEFを変更してはならない。

SESSION_BRIEF.md が存在しない場合はCodexを起動せず BLOCKER とし、テンプレートのパスと作成方法を表示してcommitを停止する。

確認項目:

- SESSION_BRIEFの目的と無関係な変更
- 対象外ファイルまたは非対象領域の変更
- 既存動作を壊すデグレの可能性
- 確定仕様との不一致
- テスト期待値の実装都合による変更
- 不要なりファクタリングや過大な差分
- セキュリティ、認証、認可、データ整合性への影響
- 判断材料不足または不確実性

review-result.schema.json は少なくとも以下を返す。

- 全体判定: OK / WARNING / BLOCKER
- 要約
- findingごとのseverityとcategory
- 対象ファイルと、特定できる場合は行番号
- 問題となる差分または挙動
- SESSION_BRIEFまたは既存仕様に照らした違反理由
- 推奨される次の対応

違反を検出した場合、Hookは結果を省略せず利用者へ表示する。

```
[AgentSkills][LLM-REVIEW][BLOCKER] Regression risk detected  
Runtime: Codex  
File: src/auth/login.ts  
Line: 84  
Category: regression  
Finding:  
  The existing role check is removed by the staged change.  
Reason:
```

```
SESSION_BRIEF authorizes redirect handling only; authorization behavior is
outside scope.
Recommended action:
  Restore the role check or update the confirmed specification before
  committing.
Commit: aborted
```

複数のfindingがある場合は、BLOCKER、WARNINGの順で全件を表示する。各findingには具体的なevidenceを必須とする。top-level statusはfindingの最大severityと一致しなければならず、矛盾した結果はFAILとしてcommitを停止する。OKの場合も、使用runtime、モデル、diff hash、context fingerprint、レビュー要約、最終結果を表示する。

判定後の動作:

- OK: commitを続行する
- WARNING: 内容を表示してcommitを停止する
- BLOCKER: 内容を表示してcommitを停止する
- Codex未導入、未認証、利用枠切れ、timeout、JSON不正: FAILとして理由を表示し、そのcommit試行を停止する

timeout値と強制終了までの猶予値は、正の整数としてローカルGit設定から読み込む。timeout時はreviewerへSIGTERMを送り、猶予時間後も終了していなければSIGKILLで停止する。これにより、応答しないreviewerがpre-commitを無期限に停止させない。

```
git config --local agentskills.reviewTimeoutSeconds 180
git config --local agentskills.reviewTimeoutKillGraceSeconds 5
```

Codexを呼び出せない場合、黙ってreviewを省略してはならない。Hookは次の2つの選択肢と実行コマンドを表示する。

1. Claude Codeの現在セッションで @subagent-review を手動実行し、OK 後に record-manual-review.sh で現在のstaged diff hashへreview結果を記録してからcommitを再実行する
2. 利用者がリスクを理解したうえで、そのcommitだけLLM reviewを明示的にskipする

表示例:

```
[AgentSkills][LLM-REVIEW][FAIL] Codex reviewer unavailable
Reason: Codex usage limit reached.

Option 1 - Review manually in Claude Code:
  @subagent-review SESSION_BRIEF.md と git diff --cached を根拠にレビューし、コード
  は変更しない。
  bash .agentskills/reviewers/record-manual-review.sh --runtime claude --status
  OK
  git commit

Option 2 - Skip LLM review once:
  AGENTSKILLS_SKIP_LLM_REVIEW=1 git commit
```

```
Mechanical gate checks will still run.
```

AGENTSILLS_SKIP_LLM_REVIEW=1 はLLM reviewだけを1回skipし、機密、巨大ファイル、空白などの機械的gateは実行する。skipはキャッシュせず、SKIP、理由、diff hashを目立つ形で表示する。git commit --no-verify は全gateを回避するため、案内しない。

OK結果は .git/agentskills/reviews/<context-fingerprint>/ に保存する。context fingerprint はstaged diffだけでなく、SESSION_BRIEF、AGENTS.md、AGENT_MODELS.md、review prompt、JSON Schema、review/gate script、リスク判定、閾値、通常reviewかescalation reviewかを含む内容 hashとする。モデル別の自動reviewとruntime別の手動reviewは別ファイルへ保存する。これらの入力が変われば同じdiffでも再レビューする。キャッシュ使用時も、その事実、diff hash、context fingerprint、元の review結果を表示する。Codexの有無を確認する前に、有効な手動review cacheを確認する。

通常reviewerと高リスク時のreviewerは AGENT_MODELS.md で設定可能にする。全commitをレビュー対象とする。初期値では、変更300行以上または10ファイル以上、高リスクパス、SESSION_BRIEF対象外、test/spec変更、モジュール横断、通常reviewerの判断不能をエスカレーション条件とする。変更量だけによる強制blockは行わない。閾値はプロジェクトごとに変更可能にする。

機密ファイル検査はstaged filesだけを対象とし、作業ツリー上での作成・編集は制限しない。削除されるファイルもblockしない。

.env と実環境用の .env.*、secrets/ 配下、*.pem、*.key、*.p12、*.pfx は階層を問わずblockする。ただし、共有用テンプレートである .env.example、.env.sample、.env.template は許可する。

出力例:

```
[AgentSkills][CHECK][BLOCKER] sensitive-files
Blocked files:
  config/.env.production
Reason:
  Sensitive files must not be committed.
Resolution:
  git restore --staged config/.env.production
```

8.2 pre-push

補助的な誤操作防止として使う。リモート側の強制機構ではなく、--no-verify で回避可能であることを前提とする。

v0.1の制御対象は、保護対象ブランチへの直接pushだけとする。force push、tag、コミット内容、テスト実行などの追加制限は行わない。

保護対象ブランチは、リポジトリ単位のGitローカル設定で指定する。

```
git config --local --add agentskills.protectedPushBranch main
git config --local --add agentskills.protectedPushBranch master
```

設定値は次のコマンドで確認する。

```
git config --local --get-all agentskills.protectedPushBranch
```

設定がない場合は main と master をデフォルトとする。プロジェクトに応じて develop、release、production などへ変更できる。

判定には現在のローカルブランチ名ではなく、pre-pushが標準入力で受け取るpush先remote refを使用する。これにより、featureブランチからの HEAD:main のようなpushも検出する。

保護対象へのpushを検出した場合は、ローカルref、remote ref、適用された設定値、block理由を表示して終了コード 1 を返す。それ以外は対象remote refと PASS を表示して終了コード 0 を返す。

強制的な保護が必要な場合は、将来GitHubのbranch protectionを併用する。

8.3 Hookのセットアップ

Git Hookはキットの配置だけでは有効化せず、配布先でユーザーまたはエージェントが .agentskills/README.md を確認し、明示的にsetup scriptを実行して導入する。AgentSkillsリポジトリ自体で作業する場合のkit rootは common/ とする。

通常実行:

```
bash .agentskills/setup/setup-hooks.sh
```

動作:

- core.hooksPathが未設定なら .agentskills/hooks を設定する
- すでに .agentskills/hooks なら変更せず PASS とする
- 別の値が設定済みなら上書きせず BLOCKER とする
- 現在値、競合理由、選択可能な対応を表示する

既存設定を理解したうえで置き換える場合だけ、次を実行する。

```
bash .agentskills/setup/setup-hooks.sh --force
```

--force 使用時は、以前の値をローカルGit設定の agentskills.previousHooksPath に保存し、変更前後の値と復旧コマンドを表示する。

setup scriptは、自身の配置場所からkit rootとGitルートからの相対パスを求める。既存のHusky、Lefthook、独自hookを自動編集または自動統合しない。Git Hookの導入判断、通常実行と --force の違い、既存hookからgateを手動で呼び出す方法を common/README.md に記載する。

setup scriptはGit、jq、およびSHA-256を計算できるsha256sum、shasum、opensslのいずれかを事前確認する。Codex CLIがない場合はsetup自体をblockせず、手動reviewまたは明示的skipが必要になることを警告する。Bash 3.2を最低互換範囲とし、mapfile、Bash 4専用の小文字変換、GNU timeoutへの必須依存を避ける。論理kit pathをagentskills.kitPathへ保存し、fallback commandは固定パスではなくこの値から生成する。

9. モデル切り替え方針

具体的なモデル名は設計書や共通ルールへ固定しない。モデル構成は時期、製品、利用可能な契約によって変化するため、役割だけを共通定義し、プロジェクトごとに任意設定できるようにする。

モデルの役割は以下とする。

- Expansion: アイディア出し、設計比較、仕様整理、大きな方針検討
- Convergence: 最小差分実装、軽微な修正、gate前確認
- Review: 親コンテキストから分離した独立レビュー
- Review escalation: 高リスクまたは通常reviewerが判断できない差分の独立レビュー
- Failure analysis: 原因不明の失敗や複雑な依存関係の分析

テンプレートは common/config/AGENT_MODELS.template.md に置く。利用先では必要に応じてプロジェクトルートに AGENT_MODELS.md を作成する。

設定形式の例:

```
| Runtime | Expansion | Convergence | Review | Review escalation | Failure  
analysis |  
|---|---|---|---|---|---|  
| Claude Code | auto | auto | auto | auto | auto |  
| Codex | auto | auto | auto | auto | auto |  
| Gemini | auto | auto | auto | auto | auto |
```

AGENT_MODELS.md が存在しない場合、または値が auto の場合は、実行環境のデフォルトモデルを使用する。設定ファイルは任意とし、存在しなくてもキットは動作する。

モデル選択時は、役割、runtime、設定値、実際に選択できたモデル、設定元を表示する。

製品または実行環境が自動モデル切り替えに対応していない場合は、切り替えたと表現してはならない。WARNING として要求モデルと現在モデルを表示し、現在モデルで継続するか、ユーザーが手動で切り替えるかを確認する。

表示例:

```
[AgentSkills][MODEL][PASS]  
Role: Review  
Runtime: Codex  
Selected model: model-a  
Source: AGENT_MODELS.md
```

Failure analysis用モデルを使用する場合も、原因分析中はコード変更を行わない。

10. 初期実装スコープ

v0.1 では以下を実装する。

```
common/  
  README.md  
  rules/  
    AGENTS.base.md  
    CLAUDE.base.md  
  prompts/  
    converge-bugfix.md  
    diff-review.md  
    subagent-review.md  
    failure-analysis.md  
  briefs/  
    SESSION_BRIEF.template.md  
  config/  
    AGENT_MODELS.template.md  
  gates/  
    pre-commit-gate.sh  
    check-sensitive-files.sh  
    check-large-files.sh  
    check-diff-basic.sh  
    check-llm-review.sh  
  lib/  
    review-common.sh  
  reviewers/  
    review-staged-diff.sh  
    record-manual-review.sh  
  schemas/  
    review-result.schema.json  
  hooks/  
    pre-commit  
    pre-push  
  setup/  
    setup-hooks.sh  
  tests/  
    run-tests.sh
```

以下は v0.1 では任意または将来対応とする。

- GitHub Actions
- Branch protection
- allowed-files.txt による厳格な対象ファイル制御
- Claude Code の正式 slash command 化
- Codex の正式 slash command 対応
- Cowork 用 .skill 化
- npm package / installer 化

11. 未決事項

v0.1の実装開始を妨げる主要な未決事項はない。以下は決定済みの方針と将来見直しの対象を記録する。

11.1 リポジトリ名

AgentSkillsに決定し、ローカルディレクトリとリモートリポジトリを変更済み。

11.2 配布方式

個人利用の標準は、AgentSkillsの common/ から利用先の .agentskills へのsymlinkとする。コピーはチーム共有または単独配布向けの代替手段とする。Git submodule、setup scriptによる展開、テンプレート生成は将来対応とする。

11.3 擬似コマンド記法

初期は @xxx とする。

将来的に Claude Code 用には slash command 化を検討する。

11.4 Git Hook の厳しさ

初期は以下とする。

- 秘密ファイル: block
- 巨大ファイル: block
- 空白エラー: block
- テストファイル変更: warning
- 目的外変更: warning

将来的に必要なであれば厳格化する。

12. 結論

既存の AgentSkills リポジトリを拡張し、common/ 配下に Claude / Codex 共通のエージェント運用キットを追加する。

この設計では、スキルの自動発動に依存せず、以下の組み合わせで収束フェーズを安定化する。

- AGENTS.md / CLAUDE.md
- SESSION_BRIEF.md
- prompts/*.md
- 擬似コマンド
- diff review
- subagent review
- shell gate
- Git Hook
- 小さい commit

v0.1 の目標は、まず Markdown と shell script を整備し、ローカルリポジトリで Codex に実装させられる状態にすることである。